

05 tree traversal

Tree Traversal Pattern

1. What is it, and when is it used?

Tree traversal is the systematic process of visiting every node in a tree exactly once, in some defined order (DFS: preorder/inorder/postorder, or BFS: level order). It's used any time a problem gives you a tree (binary tree, N-ary tree, BST, trie) and asks you to compute something over its structure — search, aggregate, transform, or validate.

2. Why is it used?

Trees are recursively defined (a tree is a node + its subtrees, which are themselves trees), so recursive/DFS traversal maps directly onto the data structure's definition, letting you express "solve for this node in terms of solutions to its children" naturally (top-down or bottom-up). BFS gives you the alternative "layer by layer" view that DFS can't easily provide, useful when order-by-depth matters.

3. Where is it used?

- Binary Search Tree validation, insertion, deletion, search
- Serialization/deserialization of trees
- Lowest Common Ancestor (LCA)
- Path sum / max path sum problems
- Level order traversal, zigzag level order, right-side view
- Diameter of a tree, balanced tree check, tree height
- Trie traversal for autocomplete/word search
- File system directory traversal (conceptually a tree)
- Expression tree evaluation (parse trees, compilers)
- Symmetric tree / mirror tree checks

4. How do you identify it's a Tree Traversal problem?

Signal phrases (2-minute test): - "Given the root of a binary tree..." - "Level order traversal" / "zigzag" / "right side view" → BFS - "Path sum" / "max path sum" / "root-to-leaf" → DFS (often postorder or backtracking) - "Lowest common ancestor" → DFS with return-value bubbling - "Serialize/deserialize" → any consistent traversal + reconstruction - "Diameter", "height", "balanced" → postorder DFS (need children's answers first) - "Validate BST" → inorder DFS (should yield sorted sequence) or bounds-passing DFS - Explicit `TreeNode { val, left, right }` class given

5. Can you identify it from constraints?

Constraint clue	Implication
"Return nodes level by level"	BFS (queue-based)
"Need minimum depth" (shortest path down to a leaf)	BFS is faster in practice — stops at first leaf found
Need parent's answer to depend on children's answer	Postorder DFS (bottom-up)
Need to pass info down to children (like path prefix)	Preorder DFS (top-down), often with backtracking
Tree could be very unbalanced / skewed (height ~ n)	Watch recursion depth — may need iterative DFS with explicit stack to avoid stack overflow
n up to 10^4 - 10^5 nodes	$O(n)$ traversal is expected, fits any single-pass DFS/BFS

6. Types / Variants

1. **Preorder (Root → Left → Right)** — good for copying/serializing a tree, top-down problems.
2. **Inorder (Left → Root → Right)** — yields sorted order for BSTs; used for BST validation/kth smallest.
3. **Postorder (Left → Right → Root)** — good for deletion, bottom-up aggregation (height, diameter, subtree sums).
4. **Level Order (BFS)** — queue-based, layer by layer; used for level averages, zigzag, right-side view, min depth.
5. **Morris Traversal** — $O(1)$ space inorder traversal using temporary threaded pointers (no stack/recursion).
6. **DFS with backtracking (path-based)** — track a path list mutated on the way down and undone on the way back up (root-to-leaf paths, path sum).
7. **Iterative traversal with explicit stack** — mirrors recursive DFS without call-stack risk.

7. Rationale / Intuition

A tree's recursive definition means most problems reduce to: **"compute the answer for this node using the answers already computed for its children."** That's exactly postorder. If instead the answer needs to flow from the root down (like carrying a running path or bound), that's preorder / top-down DFS. BFS is chosen specifically when **"distance from root" or "order visited by depth"** matters, because DFS doesn't naturally group nodes by depth without extra bookkeeping.

8. Time & Space Complexity

Traversal	Time	Space (recursion / queue)
Preorder / Inorder / Postorder (recursive)	$O(n)$	$O(h)$ — h = tree height ($O(\log n)$ balanced, $O(n)$ skewed)
Level order (BFS)	$O(n)$	$O(w)$ — w = max width, worst case $O(n)$
Morris Traversal	$O(n)$	$O(1)$
Iterative DFS (explicit stack)	$O(n)$	$O(h)$

9. Comparison with Other Patterns

Pattern	Visits order	Best for	Space
DFS (pre/in/post)	Depth-first	Path problems, subtree aggregates, recursive structure	$O(h)$
BFS (level order)	Breadth-first	Shortest path/level-based problems, min depth	$O(w)$
Union-Find	N/A (not traversal)	Connectivity/grouping without full traversal	$O(n)$
Backtracking	DFS + undo	Enumerating all paths/combinations	$O(h)$
Graph DFS/BFS	Same idea, generalized	Trees are just acyclic connected graphs — same algorithms apply, plus need a <code>visited</code> set (trees don't, since no cycles)	

10. Reusable Java Templates

10.1 Node Definition

```
class TreeNode {
    int val;
    TreeNode left, right;
}
```

```

        TreeNode(int val) { this.val = val; }
    }

```

10.2 Recursive DFS — Preorder / Inorder / Postorder

```

import java.util.*;

class DFSTraversals {
    public List<Integer> preorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        preorderHelper(root, result);
        return result;
    }

    private void preorderHelper(TreeNode node, List<Integer> result) {
        if (node == null) return;
        result.add(node.val);           // Root
        preorderHelper(node.left, result); // Left
        preorderHelper(node.right, result); // Right
    }

    public List<Integer> inorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        inorderHelper(root, result);
        return result;
    }

    private void inorderHelper(TreeNode node, List<Integer> result) {
        if (node == null) return;
        inorderHelper(node.left, result);
        result.add(node.val);
        inorderHelper(node.right, result);
    }

    public List<Integer> postorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        postorderHelper(root, result);
        return result;
    }

    private void postorderHelper(TreeNode node, List<Integer> result) {
        if (node == null) return;
        postorderHelper(node.left, result);
        postorderHelper(node.right, result);
        result.add(node.val);
    }
}

```

10.3 Level Order Traversal (BFS)

```

import java.util.*;

class LevelOrder {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
            }
        }
    }
}

```

```

        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
    result.add(level);
}
return result;
}
}

```

10.4 Postorder Bottom-Up — Diameter of Binary Tree

```

class DiameterOfTree {
    private int diameter = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }

    private int height(TreeNode node) {
        if (node == null) return 0;
        int leftHeight = height(node.left);
        int rightHeight = height(node.right);
        diameter = Math.max(diameter, leftHeight + rightHeight);
        return 1 + Math.max(leftHeight, rightHeight);
    }
}

```

10.5 DFS Backtracking — Root-to-Leaf Path Sum

```

import java.util.*;

class PathSum {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(root, targetSum, new ArrayList<>(), result);
        return result;
    }

    private void backtrack(TreeNode node, long remaining, List<Integer> path,
        List<List<Integer>> result) {
        if (node == null) return;

        path.add(node.val);
        remaining -= node.val;

        if (node.left == null && node.right == null && remaining == 0) {
            result.add(new ArrayList<>(path));
        } else {
            backtrack(node.left, remaining, path, result);
            backtrack(node.right, remaining, path, result);
        }
        path.remove(path.size() - 1); // undo - backtrack
    }
}

```

11. Quick Recognition Checklist (2-minute test)

- ☐ `TreeNode` / `Node { left, right }` class explicitly given?
- ☐ Asked about levels/depth explicitly? → BFS
- ☐ Asked about paths, subtree aggregates, height, diameter, LCA? → DFS

- ☐ Answer at a node depends on children's answers? → Postorder
- ☐ Need to pass accumulated state downward? → Preorder/backtracking

If 2+ checked → Tree Traversal (pick DFS variant vs BFS based on the specific signal).